SEGMENT BLOCKS & FREQUENCY CHALLENGE SET

Custom Laboratory Problems Focus: Multi-Element Boundary Pointer Jumps

Problem 1: Server Log Telemetry Cleaner

A data center analytics tool collects a massive sorted list of server error codes thrown during a network failure. Because multiple subsystems experience identical errors consecutively, identical error codes group together in clusters. To optimize disk storage, the administrator wants to compress this data by outputting a clean report containing only the error codes that generated **strict anomalies (appearing more than 2 times)** along with their frequency.

TASKS

1. Read an integer N representing the total logged errors, followed by a sorted array containing the numeric error codes.
2. Implement a custom **Upper Bound** function to identify the boundary point where a cluster of matching codes terminates.
3. Write a looping sequence that iterates over the unique codes by jumping pointers to the start of each new segment using your boundary function.
4. For every error code that repeats **strictly more than 2 times**, print its value and total count.

Sample Input	Sample Output
`12` `101 101 101 404 404 500 500 500 500 500 502 502`	`Error 101: 3 times` `Error 500: 5 times`

Problem 2: E-Commerce Inventory Stock Depletion Tracker

An e-commerce marketplace analyzes consumer purchase history recorded across multiple sales events. The logs contain a sorted list of unique consumer item IDs purchased. The business analyst requires a report detailing items that are in high demand and experiencing rapid stock depletion. High demand is defined as an item being purchased **exactly K times or more**.

TASKS

1. Read the total purchase log count N and the sorted item ID array. Then, read an integer K representing the minimum depletion threshold.
2. Implement a custom **Upper Bound** function to compute the continuous stretch length of an individual item segment.
3. Design a loop that computes the block lengths via index skipping to ensure the entire array is processed without linear step-by-step element tracking.
4. Output the ID of any item whose frequency is greater than or equal to K.

Sample Input	Sample Output

`10`
`11 22 22 33 33 33 44 55 55 55`
`3`
	`High Demand Item: 33`
	`High Demand Item: 55`

Problem 3: Crypto Token Whale-Trade Monitor

A blockchain data explorer records a transaction volume timeline for a newly launched utility token. The transactions are sorted in ascending order of their token values. A compliance officer tracking "Whale Activity" needs a list of transaction segments where a specific value threshold is repeated. Crucially, the compliance team wants to find values that appear **at least twice but no more than three times** to isolate algorithmic test patterns.

TASKS

1. Read the total logged transactions N and the sorted array of transaction values.
2. Implement a custom **Upper Bound** function to locate the boundary break of repeating values.
3. Apply index pointer jumping logic using the bound values to determine segment lengths efficiently.
4. Filter and print only the unique transaction values whose duplication frequency is **either 2 or 3**.

Sample Input	Sample Output
`11` `5 5 12 12 12 12 20 20 20 99 100`	`Whale Pattern Value: 5 (2 occurrences)` `Whale Pattern Value: 20 (3 occurrences)`